

A GENERAL SOFTWARE RELIABILITY GROWTH MODEL FOR A DISTRIBUTED ENVIRONMENT

P.K. Kapur

Department of Operational Research, University of Delhi, Delhi, India

V.S.S. Yadavalli¹

Department of Industrial and Systems Engineering, University of Pretoria,
South Africa

e-mail: yadavalli@postino.up.ac.za

and

Archana Kumar

Department of Operational Research, University of Delhi, Delhi, India

Key words: Distributed development environment; fault severity; non-homogeneous Poisson process (NHPP); software reliability engineering (SRE); software reliability growth model.

Summary: With growth in demand for zero defects, predicting reliability of software products is gaining importance. Software reliability models are used to estimate the reliability of a software product. Most of the reliability models available in the literature, to estimate the reliability of software, are based on the complete software development lifecycle. There are, however, large software systems which are maintained by the developer organization themselves or by a third party vendor. Such systems have been in production for a long period of time. The various details regarding the development life cycle stages are usually not known to the organizations that are responsible for the maintenance of these systems. In such a scenario, the reliability models available for the complete development life cycle cannot be applied. However, predicting the defects, in the maintenance being done for such systems, is of primary importance so as to provide confidence to the system owner and also for resource planning. This paper proposes a software reliability model that incorporates testing effort and compares Non-Homogeneous Poisson Process (NHPP) based models in a distributed development environment. The

¹ Corresponding author

proposed (NHPP) model assumes that the software system consists of a finite number of reused and newly developed sub-systems. The reused sub-systems do not involve the effect of severity of the faults on the software reliability growth phenomenon because they stabilize over a period of time, i.e. the growth is uniform, whereas the newly developed sub-system does involve the fault removal phenomena for reused and newly developed sub-systems have been modelled separately and are summed to obtain the total fault removal phenomenon of the software system. The model has been validated on several software datasets and it is shown that the proposed model fares comparatively better than the existing ones.

1. Introduction

The day-to-day operation of our society is increasingly more dependent on software-based systems and tolerance to failures of such systems is decreasing. Software engineering is not only expected to help deliver a software product of required functionality on time and within cost, it is also expected to help satisfy specific quality criteria. The most prominent one is reliability. SRE is the “applied science of predicting, measuring, and managing the reliability of software-based systems to maximize customer satisfaction.” (Lyu, 1996 and Musa et al., 1990.)

Software reliability engineering techniques play a central role in the planning and control of software development projects. In particular, it is important to document the time and nature of bug occurrences, and their correction time, throughout the design and implementation phases as well as during the formal testing phase. With such data it is possible to estimate suitable reliability models which can be used to estimate the time at which the software product will have reached a target level of reliability, or to devise methods to decrease the time.

The IEEE defines reliability as “The ability of a system or component to perform its required functions under stated conditions for a specified period of time.” To most project and software development managers, reliability is equated to correctness, that is, they look to testing and the number of “bugs” found and fixed. While finding and fixing bugs discovered in testing is necessary to assure reliability, a better way is to develop a robust, high quality product through all the stages of the software lifecycle. That is, the reliability of the delivered code is related to the quality of all of the processes and products of software development; the requirements documentation, the code, test plans, and testing (IEEE Standard 610.12, 1990).)

The terms error, fault and failure are often used interchangeably, but do have different meanings. In software, an error is usually a programmer action or omission that results in a fault. A fault is a software defect that causes a failure, and a failure is the unacceptable departure of a program operation from program requirements. When measuring reliability, we are usually measuring only defects observed and defects removed (Musa et al., 1990).

Software, however, has a different fault or error identification rate. For software, the error rate is at the highest level at integration and testing. As it is tested, errors are identified and removed. This removal continues at a slower rate during its operational use; the number of errors continually decreasing, assuming no new errors are introduced. Software does not have moving parts and does not physically wear out as hardware, but it does outlive its usefulness and becomes obsolete (Lyu, 1996).

To increase the reliability the focus must be on comprehensive requirements and a comprehensive testing plan, ensuring all requirements are tested. Focus also must be on the maintainability of the software since there will be a “useful

life” phase where sustaining engineering will be needed. Therefore, to prevent software errors, we must:

1. Start with the requirements, ensuring the product developed is the one specified, that all requirements clearly and accurately specify the final product functionality.
2. Ensure the code can easily support sustaining engineering without infusing additional errors.
3. A comprehensive test program that verifies all functionality stated in the requirements is included.

Fault is the defect in the code that can be the cause of one or more failures. Inherent fault are the faults that are associated with a software product as originally written, or modified. Faults that are introduced through fault correction, or design changes, form a separate class of modified faults. Failures can be caused by software fault, functional inadequacies in software, or user errors (for example, lacks in user’s knowledge). It is important to understand that failure and how to use errors, and their frequency, tend to relate very strongly to customer’s satisfaction and perception of the product quality. On the other hand, fault is more developer oriented, since they tend to be translated into the amount of effort that may be needed to repair and maintain the system.

Severity of a failure or fault is the impact it has on the operation of a software-based system. Severity is usually closely related to the threat the problem posed in functional (service), economic (cost) terms, or in the case of critical failures, to human life. An example of a service impact classification is: critical, major and minor failure. Severity of failures (or faults) is sometimes used to partition the operational failure data, and thus informs decisions regarding failures of a particular severity, or to weight the data used in reliability and availability calculations (Kapur, Bardhan & Shatnawi, 2002).

The purpose of software testing is to assess and evaluate the quality of work performed at each step of the software development process. Software testing is the process of executing a program with the intent of finding an error. The Testing process focuses on the logical internals of the software, assuring that all statements have been tested, and on the functional externals to conduct tests to uncover errors and ensure that defined input will produce actual results that agree with required results. Although it sometimes seems that way, the purpose of testing is not to use up all the remaining budget or schedule resources at the end of a development effort. The goal of testing is to ensure that the software performs as intended, and to improve software quality, reliability and maintainability. Testing is a major consideration in software development and maintenance. For many organizations, more time is devoted to testing than any other phase of software development. Use of well-defined testing techniques and methods minimizes the testing effort while maximizing the return on investment (Kapur et al., 1999 and Pressman, 1997).

Software testing begins with the integration-testing phase, which integrates the individual units coded and tested during the implementation phase, and ends with acceptance testing of the finished system. Acceptance testing is conducted to enable a user/customer to determine whether to accept a software product. All problems and change requests should be handled according to the Software Problem Procedure documentation. The software testing methodology encompasses the following types of testing: Integration Testing and Functional Testing, Regression Testing, System Testing and Acceptance Testing. Integration Testing and Functional Testing thoroughly test all interfaces whether new or modified. It tests each function, user input command and each command option. Regression testing is the process of testing changes to computer programs to make sure that the older programming still works

with the new changes. System testing is a phase of software testing in which developers see if there are any communications flaws – either not passing information, or passing incorrect information – between modules. Acceptance testing is based primarily upon use of case scenarios (Pressman, 1997).

Many Software Reliability Growth Models (SRGMs) (Downs & Scott, 1992) have been proved to be successful in estimating the software reliability and the number of errors remaining in the software. It has been observed that the relationship between the testing time and the corresponding number of faults removed is either Exponential or S-Shaped or a mix of the two (Bittanti et al., 1998, Goel & Okumoto, 1979, Kapur et al., 1999, Kapur & Garg, 1992 and Ohba, 1984). It has been observed that the software includes different types of faults and each fault requires different strategies and different amounts of testing effort to remove it. Ohba (1984) proposed the Hyper-exponential SRGM, assuming that software consists of different modules. Each module has its characteristics and thus the faults detected in a particular module have their own peculiarities. Therefore, the Fault Removal Rate (FRR) for each module is not the same. He suggested that the fault removal process for each module be modelled separately and that the total fault removal phenomenon is the addition of the fault removal process of all the modules. Yamada et al. (1985) proposed a modified exponential SRGM assuming the software contains two types of faults. The model is based on the observation that in the early stages of the software phase, the testing team removes a large number of simple faults (faults that are easy to remove) while the hard faults are removed in the later stages of the testing phase. Accordingly, they assumed the fault removal process to be the superposition of two NHPP. The first NHPP models the removal of the simple fault while the second models the removal of the hard faults. The FRR per remaining faults for the simple types is assumed to be

greater than that of the hard type. The total FRR at the beginning of the testing phase is equal to the Fault Removal Rate of the simple faults while it is equal to the FRR of the hard faults in the later stages of the testing phase. The time delay between the failure observation and the subsequent fault removal is assumed to represent the severity of the fault. The more severe the fault, the more the time delay (Kapur et al., 2004). The fault is classified as simple if the time delay between failure observation, fault isolation and fault removal is negligible. If there is time delay, it is classified as a hard fault. If the removal of a fault after its isolation involves an even greater time delay, it is classified as a complex fault. In the models of Ohba (1984) and Yamada et al. (1985), the assumption of constant Fault Removal Rate per remaining fault still holds for each fault type of faults. The total Fault Removal Rate per remaining fault is, however, a function of time. Kapur et al. (1992) and Kapur et al. (1999) proposed an SRGM with three types of fault. For each type, the Fault Removal Rate per remaining faults is assumed to be time independent. The first type is modelled by an Exponential model of Goel and Okumoto (1979). The second type is modelled by the Delayed S-shaped model of Yamada et al. (1984). The third type is modelled by three stages Erlang model proposed by Khoshogoftaar (Woodcock & Khoshogoftaar, 1991). The total removal phenomenon is again modelled by the superposition of the three SRGMs (Kapur et al., 1995). Later they extended their model to cater for more types of faults (Kapur, Garg & Kumar, 1999). The proposed model of this paper incorporates a power logistic function to capture the learning phenomenon during the removal phase of hard and complex fault type.

Yamada et al. (2003) have proposed an SRGM based upon NHPP for a distributed software system consisting of n reused and m newly developed software components, assuming that the cumulative number of detected faults

is an exponential curve when a software system consists of several used software components; while on the other hand, the cumulative number of faults is described by an S-shaped growth curve when the newly developed software components are used. Under this assumption the SRGM based on NHPP for a distributed development environment with testing effort can be formulated as

$$H(t) = a \left[\sum_{i=1}^n p_i \left(1 - e^{-b_i W(t)} \right) + \sum_{j=1}^m p_{n+j} \left\{ 1 - \{1 + b_{n+j} W(t)\} e^{-b_{n+j} \cdot W(t)} \right\} \right]$$

$\downarrow$
Exponential

$\downarrow$
S-shaped

(1)

where	a	Expected number of initial inherent faults
	b_i	Software failure rate per inherent fault for the i -th software module
	p_i	Weight parameter which expresses the proportion of the total testing load for the software components and ($\sum p_i = 1, p_i > 0$)
	$H(t)$	Expected number of faults identified in the time interval $(0, t]$ during testing phase
	$W(t)$	Cumulative testing effort at the end of interval $(0, t]$
	i.e.	$\int_0^t w(x) dx$

In this paper we propose a software reliability growth model with power logistic function for the distributed systems. The model is based on the Non-Homogenous Poisson Process (NHPP) and can be used to estimate and predict the reliability of the software products. The goodness-of-fit of the proposed model is compared with the Flexible Software Reliability Growth Model (Kapur et al., 1999). The Flexible Software Reliability Growth Model is a combination of the exponential model (Goel & Okumoto, 1979), the Flexible Delayed S-shaped model (Kapur et al., 2006) and the Erlang K-3 stage model (Kapur et al., 1995) with logistic function. The new proposed model provides the significant improved goodness-of-fit results. Section 2 presents the proposed models. Sections 3, 4 and 5 give the method used for

parameter estimation and criteria used for validation and evaluation of the proposed model. We conclude this paper in section 6.

2. Software Reliability Growth Modelling

2.1 Framework for Modelling

The SRGM presented in this paper is based upon NHPP. The NHPP models are based on the assumption that the software system is subject to failures at random times caused by manifestation of remaining faults in the system. Hence NHPP models are used to describe the failure phenomenon during the testing phase. The counting process $\{N(t), t \geq 0\}$ of an NHPP process is given as follows.

$$\Pr\{N(t) = k\} = \frac{(m(t))^k}{k!} e^{-m(t)}, \quad k = 0, 1, 2 \quad (2)$$

and

$$m(t) = \int_0^t \lambda(x) dx$$

The intensity function $\lambda(x)$ (or the mean value function $m(t)$) is the basic building block of all the NHPP models existing in the software reliability engineering literature. These models assume diverse testing environments like distinction between failure and removal processes, learning of the testing personnel, possibility of imperfect debugging and error generation etc. In models proposed by Yamada et al. (1986) and Trachtenberg (1990), the effect of intensity of testing effort on the failure phenomenon has been studied. Faults if present in the software are exposed when the software is run. A general framework for model development has been proposed.

2.2 Model Assumptions and notations used

2.2.1 Model Assumptions

The proposed model has the following explicit assumptions.

1. Failure observation/fault removal phenomenon is modelled by NHPP.
2. Software is subject to failures during execution caused by faults remaining in the software.
3. Software system consists of a finite number of reused and newly developed sub-systems.
4. The faults existing in the software are of three types: simple, hard and complex. They are distinguished by the amount of testing effort needed to remove them and are modelled by one-stage, two-stage and three-stage processes respectively.
5. Each time a failure is observed, an immediate (delayed) effort takes place to decide the cause of the failure in order to remove it. The time delay between the failure observation and its subsequent removal is assumed to represent the severity of faults. The more severe the fault, the more the time delay.
6. The fault removal process, i.e. the debugging process is perfect.
7. The fault removal rate of the reused components is a power function of the testing effort, whereas for newly developed it is a power logistic learning function of testing effort, as it is expected that the learning process will grow with time.
8. The expected number of faults removed in $(t, t + \Delta t)$ is proportional to the number of faults remaining to be removed.

2.2.2 Model Notations

$m, m(t)$	Expected number of faults identified in the time interval $(0, t]$ during testing phase
$e, e(t)$	Expected number of instructions executed on the software in the time interval $(0, t]$
a	Total fault content
a_i	Initial fault content of type i reused component with simple faults
a_j	Initial fault content of type j newly developed components with hard faults
a_k	Initial fault content of type k newly developed component with complex faults
b_i	Proportionality constant represents failure rate/fault isolation/fault removal rate per fault for fault type i
b_j	Proportionality constant represents failure rate/fault isolation/fault removal rate per fault for fault type j
b_k	Proportionality constant represents failure rate/fault isolation/fault removal rate per fault for fault type k
$b_j(t)$	Power Logistic learning process function, i.e. fault removal rate per fault for hard faults
$b_k(t)$	Power Logistic learning process function, i.e. fault removal rate per fault for complex faults
$m_{ir}(t)$	Mean number of faults of type i removed from reused component by time t
$m_{jf}(t)$	Mean number of failures of type j caused by newly developed component by time t
$m_{jr}(t)$	Mean number of faults of type j removed from newly developed component by time t
$m_{kf}(t)$	Mean number of failures of type k caused by newly developed component by time t
$m_{ku}(t)$	Mean number of faults of type k isolated from newly developed component by time t
$m_{kr}(t)$	Mean number of faults of type k removed from newly developed component by time t
η	A constant parameter in the logistic learning function
d	Constant

p_i	$i = 1, 2$ constant
p	Reused components having simple type of faults
q	Developed components having hard type of faults
s	Developed components having complex type of faults
$\lambda(x)$	Intensity function
$w(t)$	Testing effort intensity
$W(t)$ or W	Cumulative testing effort at the end of interval $(0, t]$, i.e. $\int_0^t w(x) dx$

$$E_i(t) = \exp(-b_i W^{d+1} / (d+1))$$

$$E_j(t) = \exp(-b_j W^{d+1} / (d+1))$$

$$E_k(t) = \exp(-b_k W^{d+1} / (d+1))$$

$$B_j = (b_j W^{d+1} / (d+1))$$

$$B_k = (b_k W^{d+1} / (d+1))$$

2.3 Modelling Testing Effort

As the time increases, testing effort also increases. If testing time becomes quite large, testing effort also becomes quite large. The testing effort rate is proportional to the testing resources available.

$$\frac{dW(t)}{dt} = \nu(t) [\alpha - W(t)] \quad (3)$$

where $\nu(t)$ is the time dependent rate at which testing resources are consumed, applied to remaining available resources. Solving equation (3) under the initial condition $W(0) = 0$.

Case 1:

If $\nu(t) = v$, (3) gives exponential type curve; then

$$W(t) = \alpha (1 - e^{-\nu t}) \quad (4)$$

The density function for the above function is $\alpha \nu e^{-\nu t}$.

Case 2:

If $\nu(t) = v \cdot t$, (3) gives Rayleigh type curve; then

$$W(t) = \alpha (1 - e^{-\nu t^2/2}) \quad (5)$$

The density function for the above function is $\alpha \nu t e^{-\nu t^2/2}$.

Case 3:

If we define

$$\frac{dW(t)}{dt} = \nu \frac{W(t)}{\alpha} [\alpha - W(t)] \quad (6)$$

On solving, the cumulative testing effort consumed in the interval $(0, t)$ is given by

$$W(t) = \frac{\alpha}{1 + l e^{-\nu t}}, \quad \text{where } W(0) = \frac{\alpha}{1 + l} \quad (7)$$

where α , ν and l are constants. This is the Logistic testing effort function.

The density function for the above function is $\frac{\alpha \nu l e^{-\nu t}}{(1 + \nu e^{lt})^2}$

Case 4:

Yamada et al. (1984) described the time dependent behaviour of testing effort expenditure by a Weibull curve while proposing a SRGM. If $\nu(t) = \nu \cdot l \cdot t^{l-1}$ (3) gives a Weibull type curve

$$W(t) = \alpha (1 - e^{-\nu t^l})$$

The density function for the above function is $\alpha \nu l t^{l-1} e^{-\nu t^l}$.

Exponential and Rayleigh curves become special cases of the Weibull curve for $l = 1$ and $l = 2$ respectively.

2.4 Modelling the Simple, Hard and Complex Faults

2.4.1 Modelling the fault removal of Reused Components

Modelling the Simple Faults It is assumed that faults in the reused components are simple faults, which can be removed instantly as soon as they are observed. Hence

$$m_{ir}(t) = a_i (1 - E_i(t)) \quad (9)$$

For derivation see appendix A.

2.4.2 Modelling the fault removal of Newly Developed Components

Software faults in the newly developed software component can be of different severity. Time required for fault removal depends on the severity of faults. The faults can either be modelled as two-stage or three-stage process according to the time lag for removal.

Modelling the Hard Faults The hard faults consume more testing effort when compared with faults of reused components. This means that the testing team will have to spend more time to analyze the cause of the failure and therefore requires greater efforts to remove them. Hence

$$m_{jr}(t) = a_j \frac{(1 - (1 + B_j) E_j(t))}{1 + \eta E_j(t)} \quad (10)$$

For derivation see appendix A.

Modelling the Complex Faults There can be components still having hard faults or complex faults. These faults can require more effort for removal after isolation. Hence they need to be modelled with greater time lag between failure observation and removal. Hence

$$m_{kr}(t) = \frac{a_k \left(1 - \left(1 + B_k + \frac{B_k^2}{2} \right) E_k(t) \right)}{1 + \eta E_k(t)} \quad (11)$$

For derivation see appendix A.

2.4.3 Modelling total fault removal phenomenon

The proposed model is the superposition of the NHPP of ‘ p ’ reused and ‘ q ’ and ‘ s ’ newly developed components. (9), (10) and (11) are mean value functions of respective NHPPs (Kapur et al., 1999). Thus, the mean value function of superposed NHPP is:

$$m(t) = \sum_{i=1}^p m_{ir}(t) + \sum_{j=p+1}^{p+q} m_{jr}(t) + \sum_{k=p+q+1}^{p+q+s} m_{kr}(t) \quad (12)$$

$$m(t) = \sum_{i=1}^p a_i (1 - E_i(t)) + \sum_{j=p+1}^{p+q} a_j \left(\frac{(1 - (1 + B_j) E_j(t))}{1 + \eta E_j(t)} \right) + \sum_{k=p+q+1}^{p+q+s} \left(\frac{a_k \left(1 - \left(1 + B_k + \frac{B_k^2}{2} \right) E_k(t) \right)}{1 + \eta E_k(t)} \right) \quad (13)$$

Case 1:

Further, we consider the simple case where a software system consists of one reused (i.e. $p = 1$) and two newly developed components (one of which contains errors represented by a two-stage error removal phenomenon while the other contains errors represented by a three-stage error removal phenomenon, i.e. $p = 1$, $q = 1$ and $s = 1$ respectively) call this case as 1 – 1 – 1. Accordingly equation (13) can be rewritten as

$$m(t) = \sum_{i=1}^1 a_i (1 - E_i(t)) + \sum_{j=2}^2 a_j \left(\frac{(1 - (1 + B_j) E_j(t))}{1 + \eta E_j(t)} \right) + \sum_{k=3}^3 \left(\frac{a_k \left(1 - \left(1 + B_k + \frac{B_k^2}{2} \right) E_k(t) \right)}{1 + \eta E_k(t)} \right) \quad (14)$$

where $a_i + a_j + a_k = a$

3. Parameter Estimation

The success of a mathematical modelling approach to reliability evaluation depends heavily upon the quality of failure data collected. The estimated values of parameters of the SRGMs are based upon these data. Hence efforts should be made to make the data collection more explicit and scientific. Usually data are collected in one of the following two ways. In the first case the times between successive failures are recorded. Though this type of data collection is more desirable, it may not be simple. Complications can arise in measuring the testing effort for each fault and it may not be very convenient to note the time at each failure report. The other easier and commonly collected data type is known as grouped data. Here testing intervals are specified and the number of failures experienced during each such interval is noted. For both these data types the method of least squares and the method of maximum likelihood have been suggested and widely used for estimation of parameters of SRGMs.

3.1 Maximum Likelihood Estimation

The Maximum Likelihood Estimation (MLE) method has been extensively adopted for estimation of parameters of SRGMs based upon NHPP (Kapur et al., 1999).

For the first type of data, suppose that estimation is to be performed at a specified time t_k , not necessarily corresponding to a failure, and with a total of m_k failures being experienced at time $t_1, t_2, \dots, t_{m_k}$. Then the likelihood function for the NHPP (14) discussed above is:

$$L = \left[\prod_{i=1}^{m_k} \lambda(t_i) \right] e^{-\int_0^{t_k} \lambda(x) dx} \quad (15)$$

The MLE of the parameters can be obtained by maximizing the likelihood function or its log likelihood function ($\log L$). If the software failure data are grouped into k points (t_i, y_i) ; $i = 1, 2, \dots, k$, where y_i is the cumulative number of failure reports at time t_i . Then the likelihood function L is given as follows:

$$L = \prod_{i=1}^k \frac{[m(t_i) - m(t_{i-1})]^{y_i - y_{i-1}}}{(y_i - y_{i-1})!} e^{-\{m(t_i) - m(t_{i-1})\}} \quad (16)$$

4. Comparison Criteria for SRGMs

The performance of SRGMs are judged by their ability to fit the past software fault data (goodness of fit).

4.1 Goodness-of-fit criteria

a. The Mean Square Fitting Error (MSE):

The model under comparison is used to simulate the fault data, the difference between the expected values, $\hat{m}(t_i)$ and the observed data y_i is measured by MSE (Kapur et al., 1999) as follows.

$$MSE = \sum_{i=1}^k \frac{(\hat{m}(t_i) - y_i)^2}{k} \quad (17)$$

where k is the number of observations. The lower MSE indicates less fitting error, thus better goodness-of-fit.

b. The Akaike Information Criterion (AIC):

The criterion is defined as $AIC = -2(\text{the value of the maximum log likelihood function}) + 2(\text{the number of parameters used in the model})$.

This index (Akaike, 1974 and Kapur et al., 1999) takes into account both the statistical goodness of fit and the number of parameters that are estimated in competing models. Lower values of AIC indicate the preferred model.

c. Coefficient of Multiple Determination (R^2):

We define this coefficient as the ratio of the sum of squares resulting from the trend model to that from the constant model subtracted from 1 (Kapur et al., 1999),

$$\text{i.e. } R^2 = 1 - \frac{\text{residual SS}}{\text{corrected SS}}. \quad (18)$$

R^2 measures the percentage of the total variation about the mean accounted for the fitted curve. It ranges in value from 0 to 1. Small values indicate that the model does not fit the data well. The larger R^2 , the better the model explains the variation in the data.

d. Prediction Error (PE):

The difference between the observation and the prediction of the number of failures at any instant of time i is known as PE_i . The lower the value of Prediction Error the better is the goodness-of-fit (Pillai & Nair, 1997).

e. Bias:

The average of the PEs is known as bias. The lower the value of Bias the better is the goodness-of-fit (Pillai & Nair, 1997).

f. Variation:

The standard deviation of PE is known as variation.

$$\text{Variation} = \sqrt{\left(\frac{1}{N-1}\right) \sum (PE_i - \text{Bias})^2}$$

The lower the value of Variation the better is the goodness-of-fit (Pillai & Nair, 1997).

g. Root Mean Square Prediction Error:

It is a measure of closeness with which a model predicts the observation.

$$RMSP E = \sqrt{(\text{Bias}^2 + \text{Variation}^2)} \quad (20)$$

The lower the value of Root Mean Square Prediction Error the better is the goodness-of-fit (Pillai & Nair, 1997).

5. Model Validation and Data Description

To check the validity of the proposed model and to find out its software reliability growth, it has been tested on two Data Sets DS-I and DS-II. SRGMs obtained through Exponential, Rayleigh, Logistic and Weibull testing effort functions have been validated on the datasets DS-I and DS-II.

DS-I

This data is cited from M. Ohba (1984). The software was tested for 19 weeks during which 47.65 computer hours were used and 328 faults were removed. The estimation results for Exponential, Rayleigh, Logistic and Weibull function are given in table 1. Based upon the estimated parameters, parameters of SRGM (14) were estimated. The Parameter Estimation result and the goodness-of-fit results for the proposed model are given in table 2. The goodness-of-fit curves for DS-I are given in figures 1–4.

DS-II

This data is cited from Musa et al. (1990). The software was a real-time command and control system, which was tested for 92 days (21 weeks). The delivered object instructions were 21,700 involving 9 programmers. 300.6 computer hours were used. 136 faults were removed during testing. The estimation results for Exponential, Rayleigh, Logistic and Weibull function are given in table 1. Based upon the estimated parameters, parameters of SRGM (14) were estimated. The Parameter Estimation result and the goodness-of-fit results for the proposed model are given in table 3. The goodness-of-fit curves for DS-II are given in figures 5–8.

Comment

The performance of an SRGM is judged by its ability to fit the past software failure/fault removal data (goodness of fit). Different distributed combinations for the proposed model were tried. The Proposed model in combination

with the Exponential Model (Goel & Okumoto, 1979) with testing effort, Flexible Delayed S-Shaped Model (Kapur et al., 2006) with testing effort and Generalised Erlang Model with logistic function (Kapur et al., 1995) with testing effort gives better result as compared to other models. For DS-I the Weibull function gives the best results for the proposed model. For DS-II the logistic function gives the best results for the proposed model. The proposed model is flexible. It reduces to the Flexible SRGM for Distributed Systems (Kapur et al., 1999).

For Case I

when testing the effort function is Rayleigh and the Dataset is DS-I then $d = 0$ (see table 2).

For Case II

when testing the effort function is Weibull and the Dataset is DS-II then $d = 0$ (see table 3).

It can be observed from the estimation results that the proposed model given by (14) gives the best estimates in terms of goodness of fit for both the data sets except for AIC (see table 2 and table 3).

Table 1. DS-I and DS-II

Testing Effort Function	Parameter estimation for DS-I			Parameter Estimation for DS-II		
	α	ν	l	α	ν	l
Exponential function	19029.34	0.0001293	–	35395.2	0.0000206	–
Rayleigh function	49.29613	0.0137013	–	22127.5	0.0000043	–
Logistic function	54.7841	0.2265891	13.0310	45.15	0.2461095	148.19
Weibull function	782.6030	0.0023787	1.11457	26.77	0.0000000102	6.38

Table 2. DS-I

Types of Testing Effort Function	Models under Comparison	Parameter Estimation						
			a	b_1	b_2	b_3	η	d
Exponential	Proposed Model	Estimated Value	438	.08	.11	.12	50.43	.131
		Asymptotic Standard Error	57.36	.017	.024	.018	10.88	.028
		Asymptotic 95% Interval						
		Lower	343	.05	.06	.08	38.23	.087
		Upper	533	.11	.16	.16	62.63	.175
Rayleigh	Proposed Model	Estimated Value	413	.41	.08	.07	3.64	0
		Asymptotic Standard Error	54.09	.088	.001	.011	.785	0
		Asymptotic 95% Interval						
		Lower	310	3	.05	.04	1.95	0
		Upper	516	.49	.11	.10	5.33	0
Logistic	Proposed Model	Estimated Value	347	.05	.13	.69	3.45	.132
		Asymptotic Standard Error	45.44	.010	.028	.118	.744	.028
		Asymptotic 95% Interval						
		Lower	246	.03	.07	.47	2.05	.088
		Upper	448	.07	.19	.91	4.85	.176

Weibull	Proposed Model	Estimated Value	432	.09	.06	.73	7.96	1.82
		Asymptotic Standard Error	56.58	.019	.013	.029	1.71	.392
		Asymptotic 95% Interval						
		Lower	340	.05	.04	.40	5.96	1.08
		Upper	524	.13	.08	1.06	9.96	2.56
Exponential	Flexible SRGM [8]	Estimated Value	363	.03	.74	.17	54.36	—
		Asymptotic Standard Error	47.54	.006	.161	.029	11.73	—
		Asymptotic 95% Interval						
		Lower	250	.018	.49	.09	39.46	
		Upper	476	.042	.99	.25	69.26	
Rayleigh	Flexible SRGM *	Estimated Value	398	.40	.03	.09	.40	—
		Asymptotic Standard Error	52.12	.086	.006	.015	.091	
		Asymptotic 95% Interval						
		Lower	263	.32	.018	.05	.31	
		Upper	533	.48	.042	.13	.49	
Logistic	Flexible SRGM *	Estimated Value	437	.007	.143	.56	25.48	—
		Asymptotic Standard Error	57.22	.001	.031	.121	5.52	—
		Asymptotic 95% Interval						
		Lower	344	.003	.072	.32	13.14	
		Upper	530	.011	.214	.80	37.82	

Weibull	Flexible SRGM *	Estimated Value	336	.06	.84	.18	84.82	–
		Asymptotic Standard Error	44.00	.012	.183	.030	18.30	
		Asymptotic 95% Interval Lower	233	.04	.46	.08	63.34	
		Upper	439	.08	1.22	.28	106.30	

* (Kapur et al., 1999)

Table 2. Continued

Types of Testing Effort Function	Models under Comparison	Comparison Criteria					
		R ²	MSE	AIC	Bias	Variation	RMSPE
Exponential	Proposed Model	.99665	34.54	195.56	0.0141	6.03862	6.03864
Rayleigh	Proposed Model	.99658	35.31	189.94	−0.0733	6.1048	6.1052
Logistic	Proposed Model	.99628	35.97	199.67	−0.0553	5.9654	5.9656
Weibull	Proposed Model	.99673	33.71	192.30	0.3551	6.3193	6.3293
Exponential	Flexible SRGM*	.99661	34.99	190.16	−0.3312	6.0678	6.0768
Rayleigh	Flexible SRGM*	.99654	35.72	187.62	−0.0676	6.1401	6.1405
Logistic	Flexible SRGM*	.99503	51.26	216.04	0.3407	6.9560	6.9644
Weibull	Flexible SRGM*	.99668	34.27	189.84	0.0722	6.0145	6.0149

* (Kapur et al., 1999)

Table 3. DS-II

Types of Testing Effort Function	Models under Comparison	Parameter Estimation						
			a	b_1	b_2	b_3	η	d
Exponential	Proposed Model	Estimated Value	238	.18	.02	.0052	6.49	1.28
		Asymptotic Standard Error	31.17	.038	.004	.001	1.400	.276
		Asymptotic 95% Interval						
		Lower	153	.12	.01	.0034	4.14	.080
		Upper	323	.24	.03	.0070	8.84	2.48
Rayleigh	Proposed Model	Estimated Value	149	.96	.05	.7147	.9154	.6382
		Asymptotic Standard Error	19.51	.207	.010	.154	.1975	.1377
		Asymptotic 95% Interval						
		Lower	92	.73	.03	.4126	.6328	.4326
		Upper	206	1.19	.07	1.0168	1.198	.8438
Logistic	Proposed Model	Estimated Value	172	.10	.07	.3005	3.88	4.74
		Asymptotic Standard Error	22.52	.021	.015	.604	.8373	.1022
		Asymptotic 95% Interval						
		Lower	108	.03	.04	.1908	2.08	.288
		Upper	236	.17	.10	.4102	5.68	.660

Weibull	Proposed Model	Estimated Value	142	.88	.31	.9731	123.6	0
		Asymptotic Standard Error	18.59	.189	.066	.209	26.67	0
		Asymptotic 95% Interval Lower	90	.58	.189	.7252	81.2	0
		Upper	194	1.18	.431	1.221	166.0	0
Exponential	Flexible SRGM*	Estimated Value	202	.008	.29	.4769	192.3	–
		Asymptotic Standard Error	26.45	.001	.062	.162	41.49	
		Asymptotic 95% Interval Lower	141	.0057	.19	.2934	139.4	
		Upper	263	.0103	.39	.6604	245.2	
Rayleigh	Flexible SRGM*	Estimated Value	144	.89	.28	.2961	15.67	–
		Asymptotic Standard Error	18.86	.192	.060	.063	3.38	
		Asymptotic 95% Interval Lower	91	.65	.16	.1950	10.31	
		Upper	197	1.13	.40	.3972	21.03	
Logistic	Flexible SRGM*	Estimated Value	148	.07	.27	.6007	29.44	–
		Asymptotic Standard Error	19.38	.015	.058	.129	6.35	
		Asymptotic 95% Interval Lower	91.7	.04	.17	.4001	18.14	
		Upper	204.3	.10	.37	.8013	40.74	

* (Kapur et al., 1999)

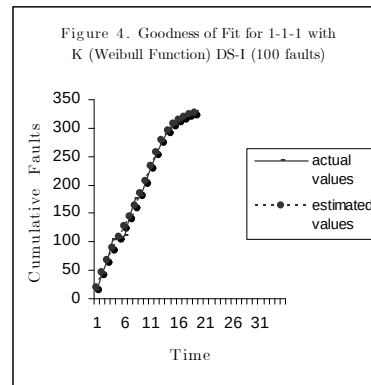
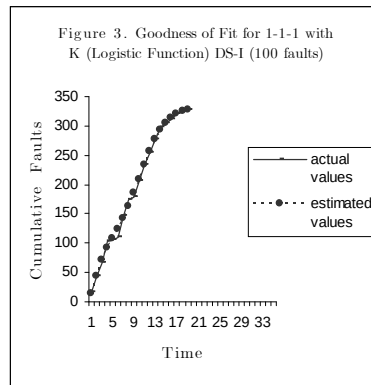
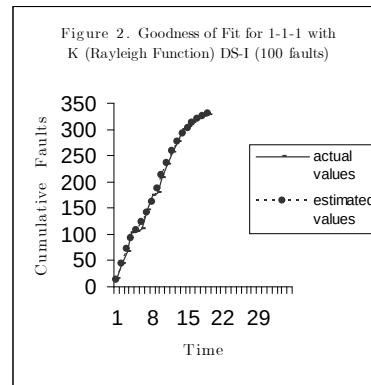
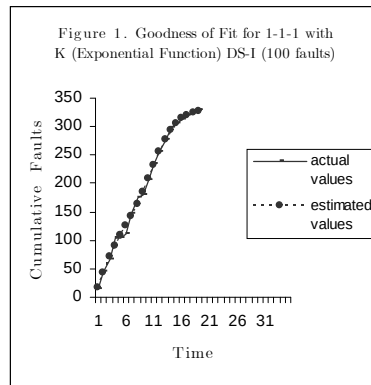
Weibull	Flexible SRGM*	Estimated Value	217	.88	.14	.8700	3.06	—
		Asymptotic Standard Error	28.42	.189	.030	.187	.6603	
		Asymptotic 95% Interval						
		Lower	147	.58	.09	.6325	2.03	
		Upper	287	1.18	.19	1.1075	4.09	

* (Kapur et al., 1999)

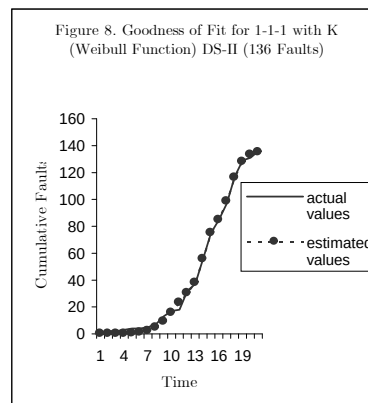
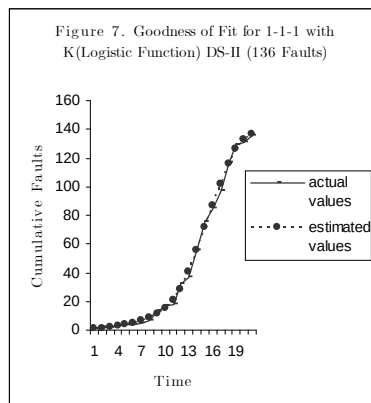
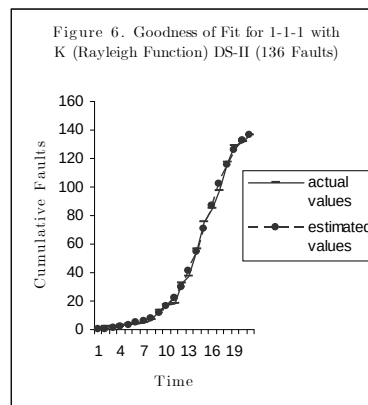
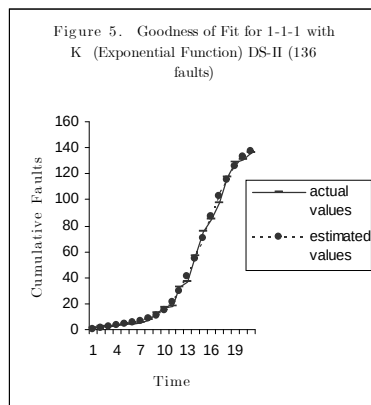
Table 3. Continued

Types of Testing Effort Function	Models under Com- parison	Comparison Criteria					
		R ²	MSE	AIC	Bias	Variation	RMSPE
Exponential	Proposed Model	.99766	5.55	116.47	-0.0878	2.4124	2.4140
Rayleigh	Proposed Model	.99765	5.58	118.91	-0.2521	2.4050	2.4181
Logistic	Proposed Model	.99809	4.53	112.65	.03401	2.1808	2.1810
Weibull	Proposed Model	.99774	5.35	98.52	-1.0290	2.1236	2.3598
Exponential	Flexible SRGM*	.99594	9.63	123.53	-0.6969	3.099	3.1765
Rayleigh	Flexible SRGM*	.99764	5.57	114.07	-0.0958	2.4190	2.4209
Logistic	Flexible SRGM*	.99795	4.86	111.32	.10446	2.2584	2.2608
Weibull	Flexible SRGM*	.99766	5.55	116.47	-0.8379	2.4650	2.6035

* (Kapur et al., 1999)

GOODNESS-OF-FIT CURVES FOR PROPOSED MODEL FOR DS-I

GOODNESS-OF-FIT CURVES FOR PROPOSED MODEL FOR DS-II



6. Conclusion

In this paper, we have presented a Software Reliability Growth Model (SRGM) defining errors of different severity with testing effort function for the distributed system. The model considers that the software system consists of a finite number of reused and newly developed sub-systems. The time delay between the failure observation and the subsequent fault removal for the proposed model represents the severity of the fault. The proposed model is compared with Flexible Software Reliability Growth Model (Kapur et al., 1999). The model takes into account different types of faults (Simple Hard and Complex). The power function of testing effort in the proposed model provides flexibility to the model and increases the reliability for the proposed model. The proposed model has been validated, evaluated and compared with the well-documented NHPP model by applying them to an actual software development project. The results of the proposed model are fairly encouraging. We feel that the proposed model depicts better results.

References

- AKAIKE, H. (1974). A new look at statistical model identification. – *IEEE Transactions on Automatic Control* **19**, 716–723.
- BITTANTI, S., BOLZERN, P., PEDROTTI, E. & SCATTOLINI, R. (1998). *A flexible modeling approach for software reliability growth*, in G. Goos & J. Harmanis, eds.: *Software Reliability Modelling and Identification*. Springer Verlag, Berlin, 101–140.
- DOWNS, T. & SCOTT, A. (1992). Evaluating The Performance Of Software Reliability Models. *IEEE Transactions on Reliability*, **41**(4): 532–538.

- GOEL, A.L. & OKUMOTO, K. (1979). Time dependent error detection rate model for software reliability and other performance measure. *IEEE Transactions on Reliability*, **R-28** (3): 206–211.
- IEEE Standard 610.12-1990. Glossary of Software Engineering Terminology.
- KAPUR, P.K., BARDHAN, A.K., SHATNAWI, O. (2002). Why software reliability growth modelling should define errors of different severity. *Journal of the Indian Statistical Association*, Vol. **40**, 2, 119–142.
- KAPUR, P.K., GOSWAMI, D.N., BARDHAN, A.K. (2006). A flexible delayed S-shaped software reliability growth model. To appear in the Journal of Computer Society of India.
- KAPUR, P.K., GARG, R.B. & KUMAR, S. (1999). (1999). Contributions to hardware and software reliability. *World Scientific*, Singapore.
- KAPUR, P.K. & GARG, R.B. (1992). A software reliability growth model for an error removal phenomenon. *Software Engineering Journal* **7**, 291–294.
- KAPUR, P.K., SHATNAWI, O., YADAVALLI, V.S.S. (2004). A software fault classification model. *South African Computer Journal*. Number 33, 1–9.
- KAPUR, P.K., YOUNES, S. & AGARWALA, S. (1995). Generalised Erlang model with n types of faults. *CSI Bulletin*.
- KAPUR, P.K., YOUNES, S. & GROVER, P.S. (1995). Software reliability growth model with errors of different severity. *Computer Science and Informatics (India)* **25**(3): 51–65.
- LYU, Michael R. (1996). *Handbook of software reliability engineering*. IEEE Computer Society Press.
- MUSA, J.D., IANNINI, A. & OKUMOTO, K. (1990). *Software reliability: measurement, prediction, applications*. Professional Edition: Software Engineering Series. McGraw-Hill, New York.
- OHBA, M. (1984). Software reliability analysis models. *IBM Journal of Research and Development*, **28**, 428–443.
- PILLAI, K. & NAIR, V.S.S. (1997). A model for software development effort and cost estimation. *IEEE Transactions on Software Engineering*; vol. **23**(8), pp. 485–497.

- PRESSMAN, Roger S. (1997) *Software engineering – A practitioner's approach* – Fourth Edition. McGraw Hill.
- TRACHTENBERG, M. (1990). A general theory of software reliability modeling. *IEEE Transactions On Reliability*, Vol. **39**(1), pp .92–96.
- WOODCOCK, T.G. & KHOSHGOFTAAR, T.M. (1991). Software reliability model selection: A case study. *Proceedings Of The International Symposium On Software Reliability Engineering*, pp. 183–191.
- YAMADA, S. HISHITANI, J. & OSAKI, S. (1993). Software reliability growth models with a Weibull test effort, a model and application. *IEEE Transactions on Reliability*, Vol. **R-42**: pp. 19–23.
- YAMADA, S. OHBA, M & OSAKI, S. (1984). S-shaped software reliability growth models and their applications. *IEEE Transactions on Reliability*, **R-33**: 169–175.
- YAMADA, S., OHTERA, H. & NARIHISA, H. (1986). Software reliability growth models with testing-effort. *IEEE Transactions On Reliability*, Vol. **R-35**, pp. 19–23.
- YAMADA, S., OSAKI, S. & NARIHISA, H. (1985). Software reliability growth models with two types of errors. *Recherche Operationnelle/Operations Research (RAIRO)*. **19**: 87–104.
- YAMADA, S., TAMURA, Y. & KIMURA, M. (2003). A software reliability growth model for a distributed development. *Environment-Electronics & Communication in Japan Part 3*, Vol. **83**, pp. 1446–53.

Appendix A

Modelling the Simple Faults

Fault removal of reused components is modelled as a one-stage process:

$$\frac{\frac{d}{dt}m_{tr}(t)}{w(t)} = b_i W^d (a - m_{ir}(t)) \quad (1)$$

For different values of d , we try to capture different types of growth curves.

The one-stage process as modelled in equation (1) describes the fault removal process.

Solving the differential equation (1) under the boundary condition $m_{ir}(t = 0) = 0$, we get:

$$m_{ir}(t) = a_i (1 - E_i(t)) \quad (2)$$

Modelling the Hard Faults

The removal process for hard faults is modelled as a two-stage process:

$$\frac{\frac{d}{dt}m_{jf}(t)}{w(t)} = b_j W^d (a_j - m_{jf}(t)) \quad (3)$$

$$\frac{\frac{d}{dt}m_{jr}(t)}{w(t)} = b_j(W) (m_{jf}(t) - m_{jr}(t)) \quad (4)$$

where

$$b_j(W) = \frac{b_j W^d}{1 + \eta E_j(t)}. \quad (5)$$

$b_j(W)$ as defined above can be termed a power logistic function. Thus $b_j(W)$ is very flexible in the sense that for $d = 1$, $b_j(W)$ is a simple logistic rate. If $\eta = 0$ (i.e. no learning) then $b_j(W)$ is the same as the fault rate for simple faults. The first stage of the two-stage process is given by equation (3). This stage describes the failure observation process. The second stage of the two-stage process given by equation (4) describes the delayed fault removal process. During this stage the fault removal rate is assumed to be time dependent. The reason for this assumption is to incorporate the effect

of learning on the removal personnel. The Power Logistic learning function is used to represent the knowledge gained by the removal team.

Solving the differential equations (3) and (4) under the boundary condition, $t = 0$, $W = 0$, $m_{jf}(t = 0) = 0$ and $m_{jr}(t = 0) = 0$, we get:

$$m_{jr}(t) = a_j \frac{(1 - (1 + B_j) E_j(t))}{1 + \eta E_j(t)} \quad (6)$$

Modelling the Complex Faults

The removal process for complex faults is modelled as a three-stage process:

$$\frac{\frac{d}{dt} m_{kf}(t)}{w(t)} = b_k W^d (a_k - m_{kf}(t)) \quad (7)$$

$$\frac{\frac{d}{dt} m_{ku}(t)}{w(t)} = b_k (W^d) (m_{kf}(t) - m_{ku}(t)) \quad (8)$$

$$\frac{\frac{d}{dt} m_{kr}(t)}{w(t)} = b_k (W) (m_{ku}(t) - m_{kr}(t)) \quad (9)$$

where

$$b_k(W) = \frac{b_k W^d}{1 + \eta E_k(t)} \quad (10)$$

The first stage of the three-stage process is given by equation (7). This stage describes the failure observation process. The second stage given by equation (8) describes the fault isolation process. The third stage given by equation (9) describes the fault removal process. During this stage the fault removal rate is assumed to be time dependent. The Power Logistic learning function is used again to represent the knowledge gained by the removal team.

Solving the differential (7) to (9) equations under the boundary condition, $t = 0$, $W = 0$, $m_{kf}(t = 0) = 0$, $m_{ku}(t = 0) = 0$ and $m_{kr}(t = 0) = 0$, we get:

$$m_{kr}(t) = \frac{a_k \left(1 - \left(1 + B_k + \frac{B_k^2}{2} \right) E_k(t) \right)}{1 + \eta E_k(t)} \quad (11)$$

Appendix B

Alternate Formulation of Proposed SRGM

The failures depend upon the number of instructions executed as well as the number of faults remaining. The number of failures executed is a power function of effort.

$$\begin{aligned} \frac{\delta}{dt} m_{ir}(t) &= \frac{dm}{de} \frac{de}{dW} \frac{dW}{dt} & (1) \\ \text{Let } \frac{dm}{de} &= p_1 (a - m), \quad \frac{de}{dW} = p_2 W^d \\ \text{So } \frac{d}{dt} m_{ir}(t) &= p_1 p_2 W^d (a - m) \frac{dW}{dt} \\ \text{Let } b_i &= p_1 p_2 \\ \frac{d}{dt} m_{ir}(t) &= b_i W^d (a - m_{ir}(t)) \frac{dW}{dt} & (2) \end{aligned}$$

Solving the differential equation (2) under the boundary condition $m_{ir}(t = 0) = 0$, we get:

$$m_{ir}(t) = a_i (1 - E_i(t)) \quad (3)$$

For Two Stage:

$$\frac{d}{dt} m_{jf}(t) = b_j W^d (a_j - m_{jf}(t)) \quad (4)$$

$$\frac{d}{dt} m_{jr}(t) = b_j (W) (m_{jf}(t) - m_{jr}(t)) \quad (5)$$

where

$$b_j(W) = \frac{b_j W^d}{1 + \eta E_j(t)} \quad (6)$$

(1)

Solving the above differential equations under the boundary condition, $t = 0$, $m_{jf}(t = 0) = 0$ and $m_{jr}(t = 0) = 0$, we get:

$$m_{jr}(t) = a_j \left(\frac{(1 - (1 + B_j) E_j(t))}{1 + \eta E_j(t)} \right) \quad (7)$$

If $d = 0$ in (7) we get the ordinary logistic function for $b_j(W)$. On solving for $b_j(W)$ (7) reduces to the Flexible Delayed S-shaped Model (Kapur, Goswami & Bardhan, 2006). If $d = 0$ and $\eta = 0$ in (7) we get $b_j(W) = b$. On solving it reduces to the Exponential Model (Goel & Okumoto, 1979). The general nature of d allows us to have greater flexibility over other widely accepted SRGMs.

Similarly we can derive the result for the three-stage model.